

DESPLIEGUE DE MODELOS DE LENGUAJE (TRANSFORMERS) COMO API REST PARA CLASIFICACIÓN DE EMOCIONES

1. Descripción del Problema

La clasificación automatizada de emociones en textos extremadamente breves y dinámicos, como los tweets, constituye uno de los desafíos más vigentes en el Procesamiento del Lenguaje Natural (NLP). El uso extensivo de modismos, ironía, abreviaturas y elementos gráficos (emojis) invalida los enfoques estadísticos tradicionales de minería de texto.

Para resolver esto, se estructura e implementa un pipeline de producción (MLOps) diseñado para categorizar publicaciones en cuatro dimensiones afectivas esenciales: anger (ira), joy (alegría), optimism (optimismo) y sadness (tristeza). El flujo cubre la integración completa desde la importación de arquitecturas preentrenadas y optimizadas alojadas en Hugging Face Hub, hasta su encapsulamiento y exposición como microservicio web de alta disponibilidad y baja latencia.

2. Arquitectura de la Red Neuronal (Transformer)

El motor de inferencia se fundamenta en un modelo de codificación basado en bloques de atención unidireccional o bidireccional (arquitecturas tipo DistilBERT o BERTweet), el cual destaca por reducir la carga paramétrica sin sacrificar la coherencia semántica profunda del texto. La red procesa las secuencias vectorizadas a través de capas de autoatención mutua antes de proyectar la información consolidada hacia una cabeza densa de clasificación lineal.

Capa / Componente	Tipo de Capa	Dimensión de Entrada	Dimensión de Salida	Activación / Operación
Tokenizador (Subword)	Embeddings dinámicos	Secuencia de texto plano	128 Tokens (Máx. fijo)	Mapeo de vocabulario único
Codificador Base	Transformer Encoder Blks	128 x Dim_Modelo	128 x Dim_Modelo	GELU / Layer Normalization
Extractor Semántico	Pooling Layer (CLS)	128 x Dim_Modelo	Dim_Modelo (768)	Tangente Hiperbólica (Tanh)
Salida (Classification Head)	Lineal Densa (Linear)	Dim_Modelo (768)	4 Clases Emocionales	Softmax (Cálculo externo)

3. Pipeline de Preprocesamiento e Inferencia

El flujo de datos estructurado asegura la consistencia de las entradas numéricas que alimentan la red neuronal mediante una serie de pasos síncronos y deterministas:

1. Tokenización y Mapeo: Conversión de la cadena de caracteres a identificadores discretos (input_ids) y construcción automática de la máscara de atención (attention_mask).
2. Truncado Estricto: Configuración fija a un máximo de 128 tokens para evitar desbordamientos de memoria en la GPU/CPU durante ráfagas densas de solicitudes.
3. Tensorización Dinámica: Formateo de las matrices hacia tensores estructurados de PyTorch, aplicando técnicas de relleno (padding=True) únicamente cuando se operan solicitudes concurrentes en lote (batch).
4. Inferencia Optimizada: Aislamiento de la fase de predicción bajo el contexto @torch.inference_mode(). Esto desactiva el grafo de gradientes e históricos de retropropagación, disminuyendo el uso de memoria VRAM y agilizando el tiempo de respuesta.
5. Distribución de Probabilidad: Transformación de los puntajes crudos (logits) a través de una función Softmax exponencial para normalizar los valores en un rango estricto de [0, 1] representativo del score de cada clase.

4. Configuración del Servidor y Despliegue

La interfaz de comunicación se implementa utilizando el framework asíncrono FastAPI, gestionado operativamente por el servidor de pasarela ASGI Uvicorn. La arquitectura del servidor garantiza que el

modelo se mantenga instanciado persistentemente en la memoria compartida del entorno, eliminando la latencia de carga inicial en peticiones subsecuentes.

Método	Ruta (Endpoint)	Entrada Estructurada (JSON)	Salida de Retorno (JSON)	Justificación Operativa
GET	/health	Ninguna	{"status", "device", ...}	Verificación automatizada del estado físico del servicio.
GET	/model	Ninguna	{"hub_repo", "labels", ...}	Auditoría de metadatos, repositorio de origen y parámetros activos.
POST	/predict	{"text": "cadena"}	{"text", "predictions"}	Clasificación en tiempo real para eventos de entrada individuales.
POST	/predict/batch	{"texts": ["lista"]}	{"results": ["..."]}	Procesamiento por lotes (límite operativo: 64 registros paralelos).

5. Consumo y Validación de la API

La validación integral del servicio en producción se efectúa de manera externa para simular el comportamiento real de un cliente de software. Aprovechando el ecosistema de Cloudspaces en Lightning.ai, se desarrolló un script ejecutable e independiente denominado client.py sustentado en la biblioteca de comunicación síncrona requests.

Este módulo interactúa directamente con la dirección URL pública generada por el plugin integrado de mapeo de puertos de la infraestructura de desarrollo, apuntando al puerto expuesto 8081. El script de control permite su ejecución parametrizada por consola, aceptando cadenas directas mediante argumentos o el procesamiento masivo de archivos planos estructurados por líneas (--batch). El cliente procesa los vectores probabilísticos devueltos por el servidor de FastAPI y renderiza representaciones visuales proporcionales dentro de la terminal de comandos, certificando de este modo la completa viabilidad operacional del ciclo MLOps implantado.